МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«Тульский государственный университет»

Институт прикладной математики и компьютерных наук

ЛАБОРАТОРНАЯ РАБОТА №8

по дисциплине

Мобильное программирование

по теме

«Работа с Activity»

Выполнил: ст. гр. 220032-11 Лаукерт К.М.

Принял: к.ф.-м.н., доцент кафедры ИБ ИПМКН Лихачев И.В.

Тула 2026

Лабораторная работа №8

по теме «Работа с Activity»

Цель работы

Изучить особенности работы с Activity в среде разработки Android Studio: создание второго экрана, переход между экранами с помощью объекта Intent, передача данных в другое Activity и возврат результата на исходный экран.

Задание

Разработать приложение, которое на первом Activity угадывает загаданное пользователем число из указанного диапазона, а на втором Activity предлагает пользователю угадать случайное число из того же диапазона.

Ход выполнения

Создано приложение «Угадай число» на основе шаблона Empty Activity. В проекте два Activity: MainActivity — компьютер угадывает число, задуманное пользователем, и GuessActivity — пользователь угадывает случайное число. Оба экрана объявлены в файле AndroidManifest.xml. У MainActivity есть intent-filter с действиями MAIN и LAUNCHER, поэтому она запускается при старте приложения; GuessActivity объявлена с атрибутом android:exported="false", так как открывается только изнутри приложения.

Все надписи вынесены в строковые ресурсы res/values/strings.xml, цвета — в res/values/colors.xml. Для второго экрана создан новый цветовой ресурс guess_background (тёмно-синий фон) и собственная тема Theme.GuessNumber.Second, назначенная GuessActivity в манифесте: она задаёт тёмный фон окна, светлые значки строки состояния и свой акцентный

цвет. Разметка обоих экранов построена на LinearLayout внутри ScrollView, поэтому содержимое доступно и на небольшом экране.

На первом экране пользователь вводит границы диапазона в поля minInput и maxInput (по умолчанию 1 и 100) и нажимает кнопку «Загадать число». Метод readRange проверяет, что оба поля заполнены целыми числами, верхняя граница больше нижней, а размер диапазона не превышает 10 000 чисел; при ошибке под полями появляется сообщение rangeError. После проверки компьютер делает первую догадку в середине диапазона и показывает панель guessPanel с номером попытки и вопросом «Вы загадали число 50?».

Кнопки «Меньше» и «Больше» сужают диапазон поиска вдвое: при ответе «Больше» нижняя граница становится равной догадке плюс один, при ответе «Меньше» верхняя граница становится равной догадке минус один, после чего вычисляется новая середина. Кнопка «Угадал!» фиксирует результат. Счётчик показывает «Попытка N из M», где M вычисляется функцией maxAttempts как число делений диапазона пополам: для ста чисел это семь попыток. Если ответы пользователя противоречат друг другу и границы диапазона пересекаются, игра завершается сообщением о том, что такого числа в диапазоне нет. После завершения игры появляется итог и кнопка «Ещё раз», возвращающая экран в исходное состояние.

Второй экран получает границы диапазона из Intent и загадывает случайное число методом Random.nextInt(min, max + 1). Пользователь вводит вариант в поле guessInput и нажимает «Проверить»; после каждой попытки выводятся подсказка «Загаданное число больше» или «Загаданное число меньше» и сузившийся диапазон в строке rangeView, а в строке attemptView увеличивается номер попытки. Вариант вне оставшегося диапазона не засчитывается как попытка. Кнопка «Сдаться» завершает игру и показывает загаданное число, а кнопка «Новая игра» появляется после победы или поражения и загадывает новое число.

Переход на второй экран выполняется по кнопке «Теперь я угадаю»: метод openGuessActivity создаёт объект Intent с указанием класса GuessActivity и кладёт в него границы диапазона, после чего вызывает startActivityForResult. Результат возвращается методом setResult: второй экран передаёт загаданное число, количество попыток и признак победы, а MainActivity в onActivityResult формирует строку итога и показывает её в карточке второго экрана. Ключи, которыми обмениваются экраны, приведены в таблице 1.

Ключ	**Тип**	**Назначение**
EXTRA_MIN, EXTRA_MAX	int	границы диапазона, передаются на второй экран
EXTRA_TARGET	int	загаданное случайное число, возвращается на первый экран
EXTRA_ATTEMPTS	int	количество попыток пользователя
EXTRA_WON	boolean	признак победы: угадал или сдался

Оба Activity сохраняют своё состояние в onSaveInstanceState и восстанавливают его в onCreate: на первом экране сохраняются границы диапазона, текущая догадка, номер попытки и тексты результатов, на втором — загаданное число, границы поиска и последняя подсказка. Благодаря этому поворот экрана не сбрасывает игру.

Ключевые методы MainActivity.kt (фрагмент листинга):

```
    /** Начинает игру: компьютер делает первую догадку в середине диапазона.
*/
    private fun startGame() {
        val range = readRange() ?: return
        minValue = range.first
        maxValue = range.second
        low = minValue
        high = maxValue
        currentGuess = (low + high) / 2
        attempts = 1
        gameActive = true
        gameFinished = false
        resultMessage = null
        render()
    }

    /** Ответ «Меньше»: загаданное число находится ниже текущей догадки. */
    private fun narrowRangeToLowerHalf() {
        if (!gameActive || gameFinished) return
        high = currentGuess - 1
        makeNextGuess()
```

```
    }

    /** Ответ «Больше»: загаданное число находится выше текущей догадки. */
    private fun narrowRangeToUpperHalf() {
        if (!gameActive || gameFinished) return
        low = currentGuess + 1
        makeNextGuess()
    }

    private fun makeNextGuess() {
        if (low > high) {
            gameActive = false
            gameFinished = true
            resultMessage = getString(R.string.result_contradiction)
            render()
            return
        }
        currentGuess = (low + high) / 2
        attempts++
        render()
    }

    /** Число попыток, которое гарантирует деление диапазона пополам. */
    private fun maxAttempts(): Int {
        var size = maxValue - minValue + 1
        var count = 0
        while (size > 1) {
            size = (size + 1) / 2
            count++
        }
        return count
    }

    /** Открывает второе Activity, передавая ему границы диапазона. */
    private fun openGuessActivity() {
        val range = readRange() ?: return
        minValue = range.first
        maxValue = range.second
        val intent = Intent(this, GuessActivity::class.java).apply {
            putExtra(GuessActivity.EXTRA_MIN, minValue)
            putExtra(GuessActivity.EXTRA_MAX, maxValue)
        }
        @Suppress("DEPRECATION")
        startActivityForResult(intent, REQUEST_GUESS)
    }

    @Suppress("DEPRECATION")
    override fun onActivityResult(requestCode: Int, resultCode: Int, data:
Intent?) {
        super.onActivityResult(requestCode, resultCode, data)
        if (requestCode != REQUEST_GUESS) return
        secondScreenMessage = if (resultCode == RESULT_OK && data != null) {
            val target = data.getIntExtra(GuessActivity.EXTRA_TARGET, 0)
            val tries = data.getIntExtra(GuessActivity.EXTRA_ATTEMPTS, 0)
            if (data.getBooleanExtra(GuessActivity.EXTRA_WON, false)) {
                getString(R.string.second_screen_result_won, target, tries)
            } else {
                getString(R.string.second_screen_result_lost, target)
            }
        } else {
            getString(R.string.second_screen_result_cancelled)
        }
```

```
        render()
    }
```

Ключевые методы GuessActivity.kt (фрагмент листинга):

```
        // фрагмент onCreate: границы диапазона приходят из Intent,
        // при первом запуске загадывается случайное число
        minValue = intent.getIntExtra(EXTRA_MIN, DEFAULT_MIN)
        maxValue = intent.getIntExtra(EXTRA_MAX, DEFAULT_MAX)
        if (maxValue <= minValue) maxValue = minValue + 1

        if (savedInstanceState == null) {
            target = randomTarget()
            currentLow = minValue
            currentHigh = maxValue
        } else {
            target = savedInstanceState.getInt(KEY_TARGET, minValue)
            attempts = savedInstanceState.getInt(KEY_ATTEMPTS, 0)
            currentLow = savedInstanceState.getInt(KEY_LOW, minValue)
            currentHigh = savedInstanceState.getInt(KEY_HIGH, maxValue)
            finished = savedInstanceState.getBoolean(KEY_FINISHED, false)
            won = savedInstanceState.getBoolean(KEY_WON, false)
            hintMessage = savedInstanceState.getString(KEY_HINT)
        }

        subtitleView.text = getString(R.string.guess_subtitle, minValue,
maxValue)

    /** Обрабатывает нажатие «Проверить» либо начинает новую игру после
финала. */
    private fun onCheckClicked() {
        if (finished) {
            startNewGame()
            return
        }
        val text = guessInput.text.toString().trim()
        val guess = text.toIntOrNull()
        if (text.isEmpty() || guess == null) {
            hintMessage = getString(R.string.error_empty)
            render()
            return
        }
        if (guess < currentLow || guess > currentHigh) {
            hintMessage = getString(R.string.error_out_of_range)
            render()
            return
        }

        attempts++
        when {
            guess < target -> {
                currentLow = guess + 1
                hintMessage = getString(R.string.hint_greater)
            }
            guess > target -> {
                currentHigh = guess - 1
                hintMessage = getString(R.string.hint_less)
            }
            else -> {
                finished = true
                won = true
                hintMessage = getString(R.string.hint_won, attempts)
                sendResult()
```

```
            }
        }
        guessInput.setText("")
        render()
    }

    private fun randomTarget(): Int = Random.nextInt(minValue, maxValue + 1)

    /** Возвращает результат игры на первый экран. */
    private fun sendResult() {
        val data = Intent().apply {
            putExtra(EXTRA_TARGET, target)
            putExtra(EXTRA_ATTEMPTS, attempts)
            putExtra(EXTRA_WON, won)
        }
        setResult(RESULT_OK, data)
    }
```

Разметка первого экрана (activity_main.xml) состоит из двух карточек: в первой поля minInput и maxInput, кнопка startButton, панель guessPanel с кнопками lessButton, greaterButton, correctButton и строка результата resultView, во второй — кнопка openGuessButton. Разметка второго экрана (activity_guess.xml) содержит подзаголовок guessSubtitle, счётчик attemptView, строку диапазона rangeView, подсказку hintView, поле guessInput и кнопки checkButton, surrenderButton и backButton. Готовый вид обоих экранов приведён на рисунках в разделе «Результат выполнения». Все описания Activity находятся в манифесте, а цвета интерфейса и фон второго экрана — в colors.xml.

Результат выполнения

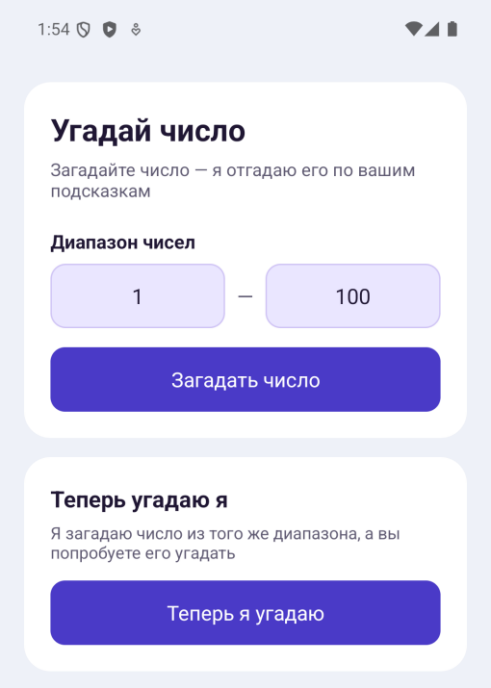


Рисунок 1 — Первый экран приложения: ввод диапазона и переход на второй экран

Приложение собрано задачей Gradle assembleDebug, установлено на эмулятор Android и запущено. На первом экране задан диапазон от 1 до 100, задумано число 37, после чего нажата кнопка «Загадать число».

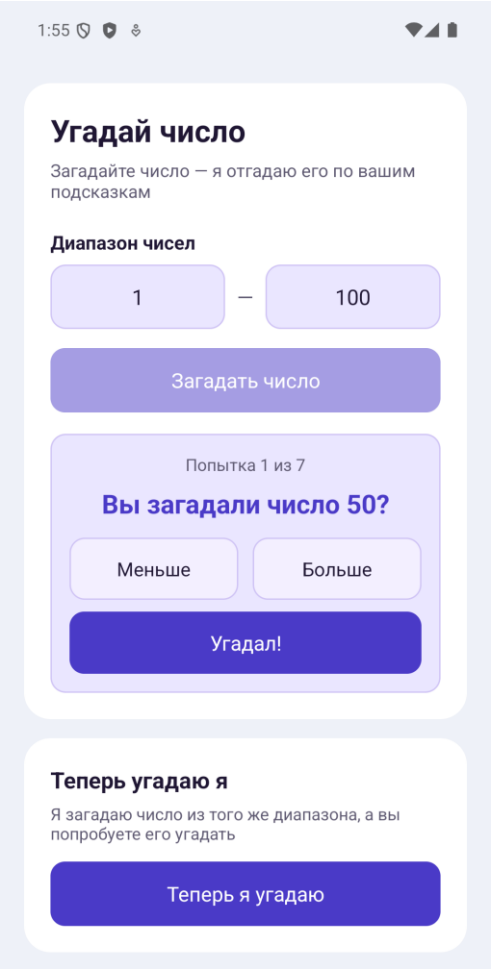


Рисунок 2 — Первая догадка компьютера — середина диапазона: попытка 1 из 7

Так как задуманное число меньше 50, нажата кнопка «Меньше». Следующей догадкой стала середина оставшегося диапазона 1–49, число 25, после ответа «Больше» диапазон сузился до 26–49, и компьютер предложил число 37. Нажатие кнопки «Угадал!» завершило игру.

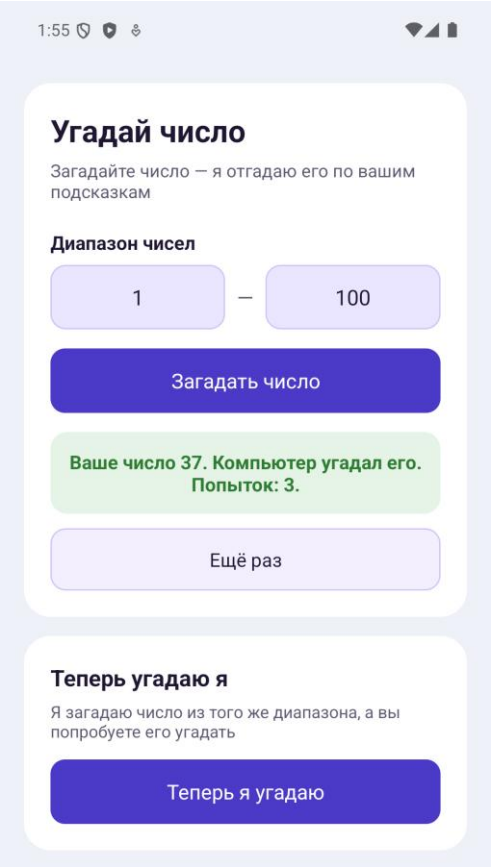


Рисунок 3 — Итог работы первого экрана: число угадано за три попытки

Сообщение «Ваше число 37. Компьютер угадал его. Попыток: 3.» подтверждает, что деление диапазона пополам укладывается в семь попыток, отведённых для ста чисел. Введённые границы после завершения игры снова доступны для изменения, а кнопка «Ещё раз» начинает новую игру.

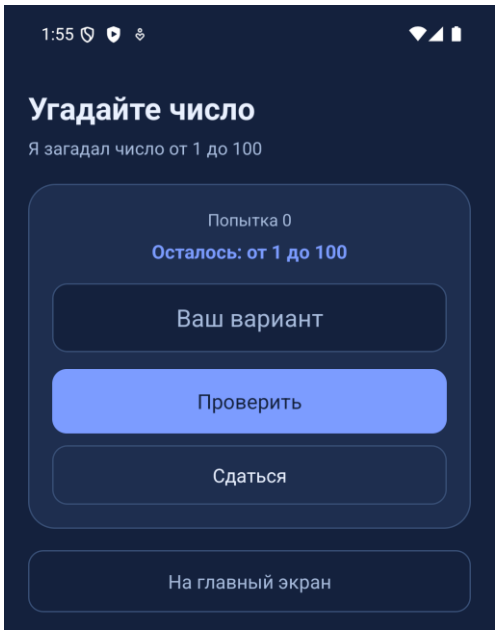


Рисунок 4 — Второй экран: пользователь угадывает случайное число из диапазона 1–100

Кнопка «Теперь я угадаю» открыла второе Activity. Подзаголовок «Я загадал число от 1 до 100» совпадает с диапазоном, введённым на первом экране, — значит, границы дошли до второго экрана в объекте Intent.

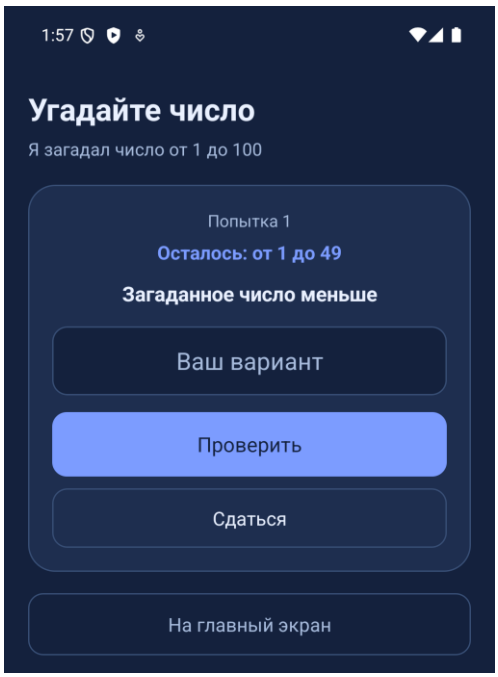


Рисунок 5 — Подсказка после первой попытки: загаданное число меньше 50, диапазон сузился до 1–49

После каждой попытки экран выводит направление поиска и оставшийся диапазон, а номер попытки увеличивается. На снимке видно, что после варианта 50 подсказка «Загаданное число меньше» сузила диапазон до 1–49 и счётчик попыток стал равен единице.

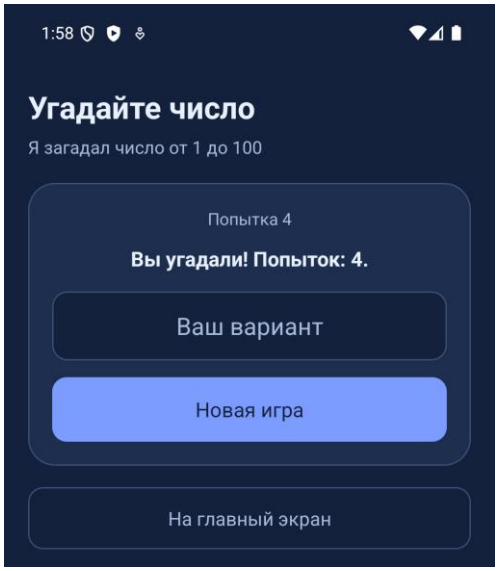


Рисунок 6 — Победа на втором экране: число угадано за четыре попытки

Загаданное число 18 найдено за четыре попытки: экран показал сообщение «Вы угадали! Попыток: 4.», скрыл подсказку и заменил кнопку «Проверить» кнопкой «Новая игра».

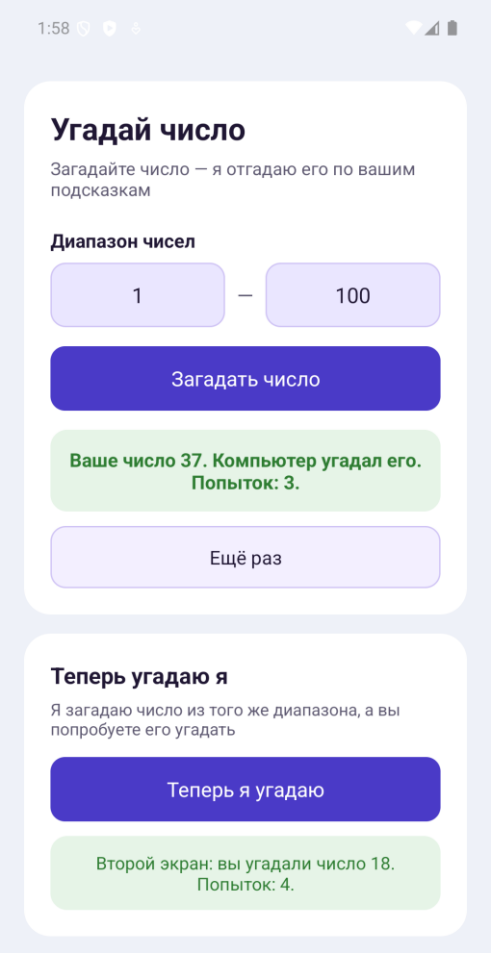


Рисунок 7 — Результат, возвращённый вторым экраном, отображается на первом экране

После нажатия кнопки «На главный экран» в карточке «Теперь угадаю я» появилось сообщение «Второй экран: вы угадали число 18. Попыток: 4.». Это подтверждает, что результат игры вернулся через setResult и был обработан в onActivityResult первого Activity.

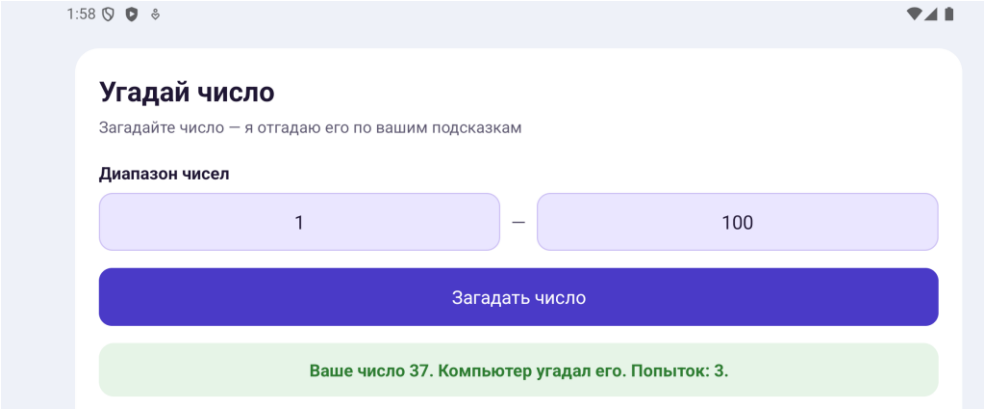


Рисунок 8 — Состояние первого экрана после поворота: введённый диапазон и результат сохранились

При повороте экрана Activity пересоздаётся. На снимке в альбомной ориентации видно, что введённые границы 1–100 и сообщение о результате остались на месте: их восстановил onCreate из объекта Bundle, переданного в onSaveInstanceState.

Вывод

В ходе работы изучены особенности работы с Activity в Android Studio. Создано приложение из двух экранов: первый угадывает задуманное пользователем число делением диапазона пополам, второй загадывает случайное число и подсказывает пользователю направление поиска. Освоены объявление Activity в манифесте, собственные темы и цветовые ресурсы для отдельного экрана, переход между экранами через объект Intent с передачей параметров, возврат результата методами setResult и onActivityResult, а также сохранение состояния Activity при пересоздании с помощью onSaveInstanceState. Проверено, что при диапазоне из ста чисел компьютер находит задуманное число не более чем за семь попыток, а данные, переданные между экранами в обе стороны, отображаются корректно.